
// TABLE OF CONTENTS

01 Computer Architecture

02 CPU Fundamentals

03 Memory Fundamentals

04 Storage Systems

05 Operating Systems

06 Processes & Threads

07 File Systems

08 System Boot Process

09 Applications & Services

10 Virtualization Basics



01 · Computer Architecture

How all the parts of a computer are organized and work together

// WHAT IS IT?

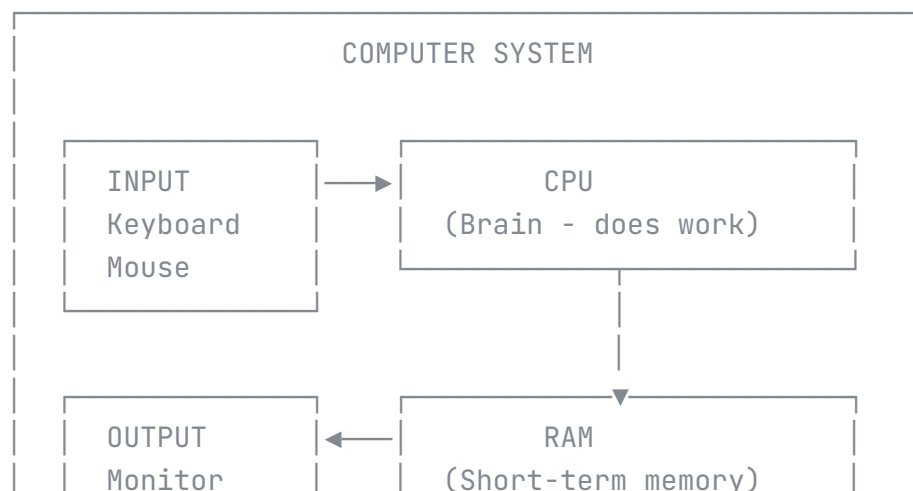
Computer Architecture is the **blueprint** of a computer — it describes how all the parts (CPU, memory, storage, input/output) are connected and talk to each other.

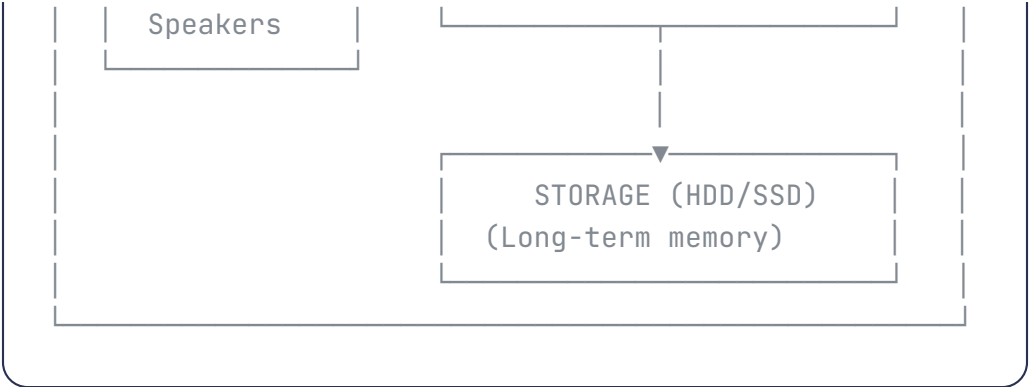


REAL-LIFE ANALOGY

Think of a computer like a **pizza restaurant**:

- The **Chef** = CPU (does all the cooking/work)
- The **Kitchen Counter** = RAM (holds ingredients currently being used)
- The **Storage Room** = Hard Drive (stores all ingredients long-term)
- The **Menu Board** = Input (tells the chef what to make)
- The **Pizza Box** = Output (delivers the result to you)





Key Components

PART	WHAT IT DOES	EXAMPLE
CPU	Executes instructions	Intel Core i7, AMD Ryzen
RAM	Temporary fast storage	8 GB, 16 GB
Storage	Permanent data storage	512 GB SSD
Motherboard	Connects everything	The main circuit board
GPU	Handles graphics	NVIDIA RTX 4090

FUN FACT

The first computer (ENIAC, 1945) weighed **30 tons** and filled an entire room. Your smartphone today is **1 million times more powerful!**



02 · CPU Fundamentals

The brain of the computer — does all the calculations

// WHAT IS IT?

CPU stands for **Central Processing Unit**. It's the chip that reads instructions and does the actual computing — math, decisions, moving data around.



REAL-LIFE ANALOGY

The CPU is like your **brain**. When you solve $5 + 3$, your brain:

1. **Fetches** the problem (reads " $5 + 3$ ")
2. **Decodes** it (understands it's addition)
3. **Executes** it (calculates the answer = 8)
4. **Stores** the result (writes down 8)

The CPU does the exact same 4 steps — billions of times per second!

THE CPU FETCH-DECODE-EXECUTE CYCLE

```
1. FETCH
  "Get the next instruction
   from memory"
  ↓
2. DECODE
  "Figure out what the
   instruction means"
  ↓
3. EXECUTE
  "Actually do the work
   (add, compare, jump...)"
  ↓
4. WRITE BACK
  "Save the result"
  ↓
← repeat forever →
```

INSIDE A MODERN CPU:

Core 1 (works)	Core 2 (works)	Core 3 (works)	Core 4 (works)	← Each core is a separate worker
Cache Memory				← Super fast memory pad inside the CPU
L1 (tiny, fastest)				
L2 (bigger, fast)				
L3 (biggest, still fast)				
Control Unit				← Traffic cop
ALU (Arithmetic Logic Unit)				← Calculator

Important Terms

- ▶ **Clock Speed (GHz)** — How many cycles per second. 3 GHz = 3 billion cycles/sec. Faster = better.
- ▶ **Cores** — Each core is like a separate worker. More cores = do more things at once.
- ▶ **Cache** — Tiny, ultra-fast memory built inside the CPU to avoid slow trips to RAM.
- ▶ **Threads** — Each core can handle 2 threads (like having 2 hands per worker). 4 cores = 8 threads.



EXAMPLE

When you open a browser and type a search:

- CPU fetches your keystrokes → decodes them as text → processes the network request → renders the page

All of that happens in **milliseconds**!



03 · Memory Fundamentals

Where data lives while the computer is working

// WHAT IS IT?

Memory (RAM) = Random Access Memory. It's the computer's short-term workspace — fast but temporary. When you turn off the computer, RAM is wiped clean.



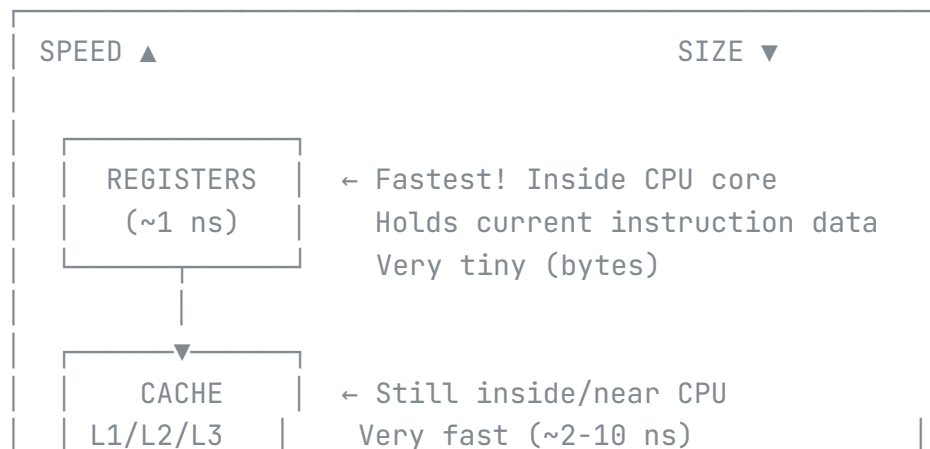
REAL-LIFE ANALOGY

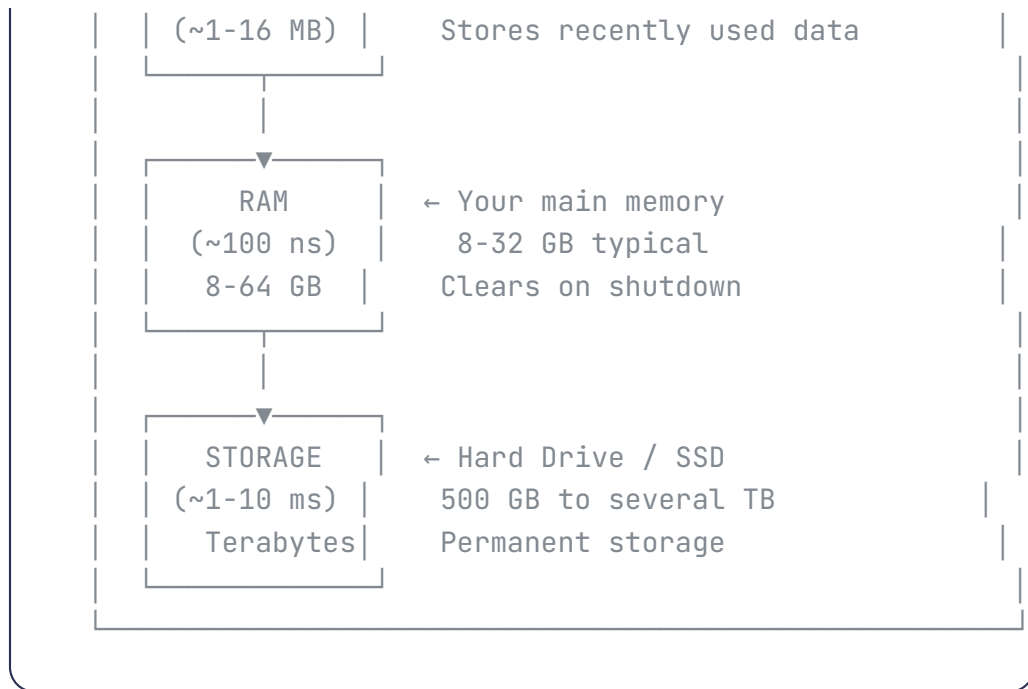
Think of memory like a **desk**:

- **RAM** = the desk surface (you put stuff here while working, cleared when you go home)
- **Hard Drive** = the filing cabinet (stores things permanently)
- **CPU Cache** = your hand/pocket (closest, fastest to access)

More RAM = bigger desk = work on more things at once!

MEMORY HIERARCHY (from fastest to slowest):





✓ RAM IS FAST
BECAUSE ...

It uses electricity to store data — no moving parts. Reading is near-instant.

✗ RAM LOSES DATA
BECAUSE ...

It's "volatile" — it needs power to keep data. Turn off power → data gone!

How RAM Works

RAM is like a giant grid of tiny switches. Each switch = 1 bit (0 or 1). A group of 8 bits = 1 byte. The CPU can jump directly to any address in RAM — that's why it's called **Random Access**.

💡 FUN FACT

In 1981, Bill Gates said "640 KB ought to be enough for anybody." Today, a simple browser tab uses **more than 640 KB!**



04 · Storage Systems

Where data is kept permanently — even with power off

// WHAT IS IT?

Storage stores data **permanently**. Your photos, games, documents — all live on storage. Unlike RAM, turning off power doesn't erase it.



REAL-LIFE ANALOGY

Storage is like a **bookshelf or filing cabinet**:

- You put books (files) on the shelf when not reading them
- When you need a book, you take it out (load into RAM) to read it
- The books stay on the shelf even when you leave the room!

Types of Storage

HDD — Hard Disk Drive



Spinning disc
Magnetic needle

← Like a vinyl record
← Reads/writes magnetically

Speed: ~100-150 MB/s Price: Cheap Size: 1-8TB
Good for: storing lots of data cheaply

SSD — Solid State Drive



Flash chips
Electric cells

← No moving parts!
← Like USB drive, but fast

Speed: ~500-7000 MB/s Price: Moderate Size:1-4TB
Good for: OS, apps – blazing fast

NVMe SSD – Fastest Consumer Storage

⚡⚡ M.2 slot
PCIe interface

← Sits directly on
motherboard!

Speed: ~3000-7000 MB/s Much faster than SATA

TYPE	SPEED	DURABILITY	PRICE	BEST FOR
HDD	Slow	Fragile (moving parts)	Cheapest	Bulk data
SATA SSD	Fast	Very durable	Moderate	OS + Apps
NVMe SSD	Blazing	Very durable	Higher	Gaming, Pro work
USB Drive	Slow/Med	Portable	Cheap	Transferring files

EXAMPLE: SAVING A PHOTO

You take a photo → it's in RAM (temporary) → you click "Save" → the OS writes it to your storage drive → now it stays there forever, even after restart.



05 · Operating Systems

The manager software that runs everything

// WHAT IS IT?

An **Operating System (OS)** is the master software that manages all hardware and software on a computer. It sits between your apps and the hardware, making sure everything works together.



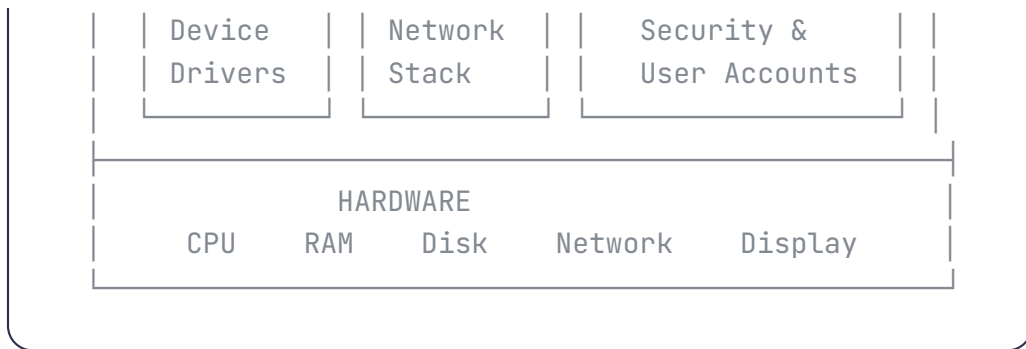
REAL-LIFE ANALOGY

The OS is like the **principal of a school**:

- Students (apps) need resources (classrooms = CPU, paper = memory)
- The principal (OS) assigns classrooms, manages schedules, resolves conflicts
- Students don't directly grab classrooms — they ask the principal!

THE OS LAYER MODEL:





Popular Operating Systems

OS	MADE BY	USED ON	FAMOUS FOR
Windows 11	Microsoft	PCs, Laptops	Most common desktop OS
macOS	Apple	Mac computers	Design, smooth UI
Linux	Community	Servers, Dev PCs	Free, open-source, powerful
Android	Google	Phones, Tablets	Most used mobile OS
iOS	Apple	iPhone, iPad	Smooth, secure

What Does the OS Do?

- ▶ **Process Management** — runs apps, gives each app CPU time
- ▶ **Memory Management** — decides which app gets how much RAM
- ▶ **File Management** — organizes your files into folders on disk
- ▶ **Device Drivers** — talks to hardware (printer, keyboard, GPU)
- ▶ **Security** — controls who can access what (user accounts,

permissions)

- ▶ **Networking** — manages internet connection, sending/receiving data



06 · Processes & Threads

How the OS runs multiple programs at the same time

// WHAT IS IT?

A **Process** is a running program. A **Thread** is a smaller unit of work inside a process. One process can have many threads working at the same time.

🍔 REAL-LIFE ANALOGY

Imagine a **burger restaurant**:

- Each **order** = a Process (running instance of "make a burger")
- Each **worker** on that order = a Thread (one grills patty, one toasts bun, one adds toppings)

Multiple workers (threads) on one order (process) = faster burger!

PROCESS vs THREAD:

PROCESS: Chrome Browser

Own Memory Space | Own files | Own PID

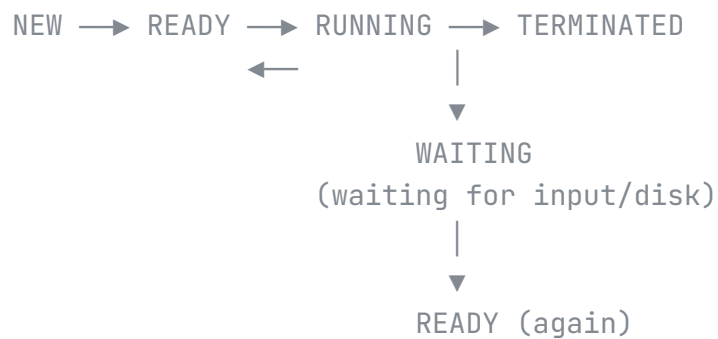
Thread 1: Rendering the webpage

Thread 2: Downloading images

Thread 3: Playing audio

(All threads share the same memory)

PROCESS STATES:



Key Differences

FEATURE	PROCESS	THREAD
Memory	Own separate memory	Shares process memory
Creation time	Slow (heavy)	Fast (lightweight)
Communication	Complex (IPC needed)	Easy (shared memory)
Crash impact	Only that process dies	Can crash entire process
Example	Opening Chrome	Each Chrome tab

EXAMPLE: PLAYING A GAME

When you play Minecraft:

- **Process:** The entire Minecraft app
- **Thread 1:** Rendering graphics (drawing the world)
- **Thread 2:** Playing background music
- **Thread 3:** Saving your world to disk

All happening simultaneously!



07 • File Systems

How the OS organizes and finds data on storage

// WHAT IS IT?

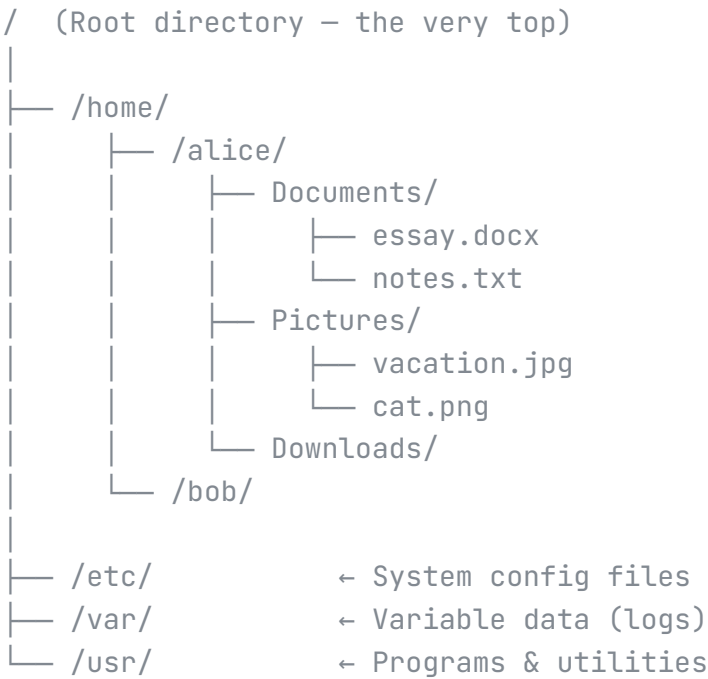
A **File System** is the method the OS uses to organize, store, find, and manage files on a disk. Without it, data would just be a sea of random 0s and 1s with no structure.

REAL-LIFE ANALOGY

A file system is like a **library catalog system**:

- **Files** = individual books
 - **Folders/Directories** = shelves and sections
 - **File System** = the catalog that tells you where every book is
- Without the catalog, finding a book would mean searching every shelf randomly!

FILE SYSTEM TREE STRUCTURE:



Each file has METADATA stored with it:

essay.docx

Size: 45 KB

Created: 2024-01-15

Modified: 2024-03-22

Owner: alice

Permissions: -rw-r--r--

Location on disk: block #48291

Common File Systems

FILE SYSTEM	USED BY	MAX FILE SIZE
NTFS	Windows	16 EB (huge!)
ext4	Linux	16 TB
APFS	macOS	8 EB

FAT32	USB drives	4 GB limit
exFAT	USB/SD cards	128 PB

💡 FUN FACT

When you "delete" a file, the OS doesn't erase the data immediately — it just removes the catalog entry! The data stays until overwritten. That's why file recovery software works!



08 · System Boot Process

What happens from pressing power button to the desktop appearing

// WHAT IS IT?

Booting is the process of starting a computer from zero — loading the OS into RAM so it's ready to use. From the moment you press the power button to when you see the desktop, a LOT happens in seconds!

☕ REAL-LIFE ANALOGY

Boot

ing is like **opening a restaurant in the morning**:

1. Unlock the door (Power ON)
2. Check everything works — ovens, fridges (POST: hardware test)
3. Get the recipe book (load BIOS/UEFI)
4. Hire the staff for the day (load OS kernel)

5. Set up all the stations (load drivers & services)
6. Open for customers (Login screen!)

BOOT SEQUENCE – STEP BY STEP:

1. POWER ON
Electricity flows to motherboard
↓
2. POST (Power-On Self Test)
CPU tests: RAM ✓ Keyboard ✓ GPU ✓
"Is all hardware working?"
(That beep you hear = POST complete)
↓
3. BIOS / UEFI
Firmware stored on motherboard chip
Finds bootable device (SSD, USB, etc.)
Looks for the bootloader
↓
4. BOOTLOADER (e.g. GRUB on Linux)
Small program that loads the OS
May show menu to choose which OS
↓
5. OS KERNEL LOADS
Core of the OS loads into RAM
Sets up memory management, processes
↓
6. INIT / systemd (Linux) or WinInit (Windows)
Starts all background services
Network, audio, display manager...
↓
7. LOGIN SCREEN / DESKTOP
Ready to use! 🎉

BIOS vs UEFI

FEATURE	BIOS (OLD)	UEFI (NEW)

Speed	Slower boot	Faster boot
Disk size support	Max 2 TB	Over 9 ZB
Interface	Text only	Mouse + GUI
Security	Basic	Secure Boot



09 · Applications & Services

The programs you use and the background workers keeping things running

// WHAT IS IT?

Applications are programs users interact with (browser, games, Word). **Services** (or daemons on Linux) are background programs that run silently to keep the system working — like Wi-Fi management, print spooler, antivirus, etc.



REAL-LIFE ANALOGY

Think of a hotel:

- **Applications** = guests using the rooms (visible, interactive)
- **Services** = hotel staff working behind the scenes: housekeeping, front desk, security, chef — running 24/7 so guests have a good experience!

APPLICATION vs SERVICE:

FOREGROUND (you interact):

Chrome Browser
VS Code
Spotify
Discord
Notepad
Calculator

BACKGROUND (invisible work):

Wi-Fi Manager
Windows Update
Antivirus Scanner
Print Spooler
Clock Sync (NTP)
SSH Daemon

APPLICATION LAYERS:

User Interface (what you see)

Application Logic (how it works)

System APIs (talks to OS)

Operating System

Hardware

Types of Applications

- ▶ **Native Apps** — Built for a specific OS (e.g., .exe for Windows, .app for macOS)
- ▶ **Web Apps** — Run in a browser (Google Docs, Gmail, YouTube)
- ▶ **Mobile Apps** — For phones (Instagram, Maps, WhatsApp)
- ▶ **System Apps** — Part of the OS (File Explorer, Terminal, Task Manager)



EXAMPLE: SERVICES ON YOUR PC RIGHT NOW

Open Task Manager → Services tab on Windows. You'll see dozens of services like:

- **Spooler** — manages printing
- **DHCP Client** — gets your IP address from router
- **Windows Defender** — scans for viruses in background



10 · Virtualization Basics

Running multiple computers inside one computer

// WHAT IS IT?

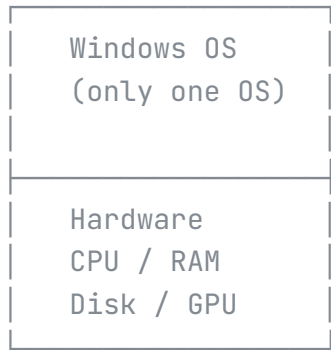
Virtualization lets you run multiple operating systems on one physical machine at the same time. Each one thinks it has its own hardware — but they're all sharing the real hardware underneath.

REAL-LIFE ANALOGY

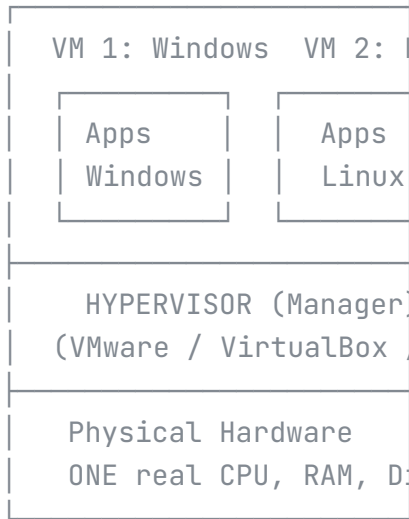
Imagine one big building (physical computer) divided into **separate apartments** (virtual machines):

- Each apartment has its own door, kitchen, bedroom (OS, apps, files)
- The tenants don't know they're sharing walls, pipes, electricity (CPU, RAM, disk)
- The **building manager (Hypervisor)** controls everything and keeps them separate!

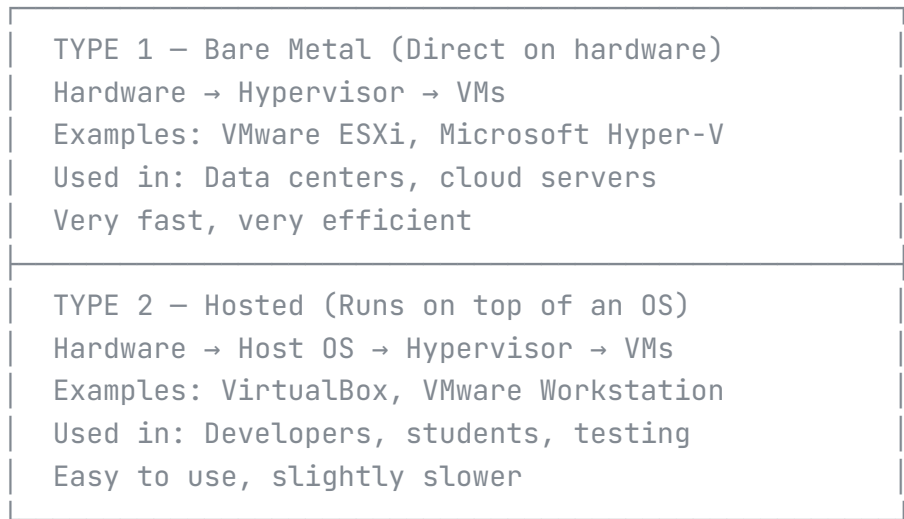
WITHOUT VIRTUALIZATION:



WITH VIRTUALIZATION:



TYPES OF HYPERVISORS:



Containers vs VMs

FEATURE	VIRTUAL MACHINE (VM)	CONTAINER (DOCKER)
What it includes	Full OS + Apps	App + dependencies only
Size	GBs	MBs
Start time	Minutes	Seconds

Isolation	Very strong	Process-level
Use case	Run different OS	Deploy apps fast

Why Virtualization Matters

- ▶ **Cloud Computing** — AWS, Azure, GCP are just millions of VMs on massive hardware
- ▶ **Cost savings** — One server does the job of 10 (less hardware needed)
- ▶ **Testing** — Developers test on VMs without risking their main system
- ▶ **Snapshots** — Save VM state like a game save — restore anytime!

FUN FACT

When you use AWS, Google Cloud, or Azure — you're renting a **Virtual Machine** running inside a massive data center. The physical server might be shared by hundreds of different customers simultaneously!

